Task 1: Blue-Green Deployment Simulation

🎯 **Objective**

Simulate a **zero-downtime upgrade** by deploying two RDS instances and gradually shifting traffic.

✅ **Steps**

1. Create Two RDS Instances

- **Blue (Current Production):**
  - Engine: MySQL **8.0**
  - Size: db.t3.micro (for lab, low cost)
  - Storage: GP2 or GP3 (20 GB min)
- **Green (New Version):**
  - Engine: MySQL **8.4**
  - Same VPC, Subnet group, Security group as Blue.
  - Set publicly accessible = Yes (for testing with MySQL Workbench).

---

2. Configure Read-Only Permissions on Green

- On **Green DB**, create a read-only user:

```
CREATE USER 'report_user'@'%' IDENTIFIED BY 'StrongPass#123';
GRANT SELECT ON *.* TO 'report_user'@'%';
FLUSH PRIVILEGES;
```

This ensures **queries can run, but no writes allowed**.

---

3. Route Traffic Using Route 53 / ALB

- **Option A: Route53 Weighted Routing**
  1. Create a **Hosted Zone** in Route53.
  2. Add two **A records (or CNAME)**:
     - db.example.com → Blue (weight 80)
     - db.example.com → Green (weight 20)
  3. Set TTL = low (e.g., 60 sec) for faster switching.

- **Option B: Application Load Balancer**
    1. Create an **ALB**.
    2. Add two **target groups**:
        - TG-Blue → Blue RDS endpoint.
        - TG-Green → Green RDS endpoint.
    3. Set **weighted target group routing** (80:20).

---

4. Validate Before Full Switch

- Test queries via the shared DNS (db.example.com).
- Confirm:
    - **Writes** go to Blue only.
    - **Reads** work on both.
- Monitor:
    - Query response time.
    - Error logs (ensure schema compatibility).

Once stable → **increase Green to 100% traffic**.

Finally, **decommission Blue** (or keep as rollback).

⚡ **Pro Tip**: AWS now has a **native RDS Blue/Green Deployment feature** that automates this process (even schema changes) → But doing it manually helps you **understand the mechanism**.

(note: full screen shots were not took because of internet issues with aws console)

Task 2: RDS Snapshot & Point-in-Time Recovery

Objective

Learn how to protect, restore, and export RDS data using **snapshots**, **PITR**, and **export tools**.

Steps

1. Create a Manual Snapshot

- Go to **AWS Console → RDS → Databases**.
- Select your DB instance → **Actions → Take Snapshot**.
- Name it: orders-snapshot-before-delete.
- Wait until status = available.

---

2. Simulate Data Loss (Delete a Table)

Connect with MySQL Workbench or CLI:

```
USE mydb;
-- Check existing tables
SHOW TABLES;
-- Delete one table intentionally
DROP TABLE orders;
```

Confirm:

```
SHOW TABLES;
```

The orders table should now be gone.

3A. Restore from Snapshot

- In **RDS Console**, select your snapshot.
- Click **Restore Snapshot → New DB instance** (e.g., restore-snapshot-db).
- After it's up, connect & confirm the orders table is back.
- Downside: It creates a **new DB instance**. You'll need to **migrate restored data** back into production manually.

3B. Point-in-Time Recovery (PITR)

- In **RDS Console**, select your DB → **Actions → Restore to point in time**.
- Choose **"Custom"** → set timestamp **just before deletion**.
- Restore into a **new DB instance** (pitr-db).
- Connect & confirm the deleted table exists.
- Advantage: You don't need snapshots every second; **PITR can rewind** to any second in retention period (default = 7 days).

---

4. Export Data

- **Option A: mysqldump (SQL)**

  Run on your laptop (if DB is publicly accessible):

```
mysqldump -h <db-endpoint> -u <username> -p mydb > mydb_backup.sql
```

This exports schema + data into .sql.

- **Option B: JSON Export**

  You can dump per-table into JSON:

```
mysql -h <db-endpoint> -u <username> -p -e "SELECT * FROM orders" mydb --batch --skip-column-names | jq -R -s -c 'split("\n") | map(select(length > 0) | split("\t"))' > orders.json
```

- **Option C: AWS DMS (Enterprise way)**
  - Set **RDS as source** & **S3 as target**.
  - Choose JSON or Parquet output format.
  - Run full load migration → data will appear in S3.

Pro Tip:

- **Snapshot restore** = good for full DB rollback.
- **PITR** = good for precise recovery (accidental delete).
- **mysqldump/DMS** = good for **offsite backups** or **migration**.

(note: screen shots were not took because of internet issues with aws console)

Task 3: Automated Backups Configuration

Objective

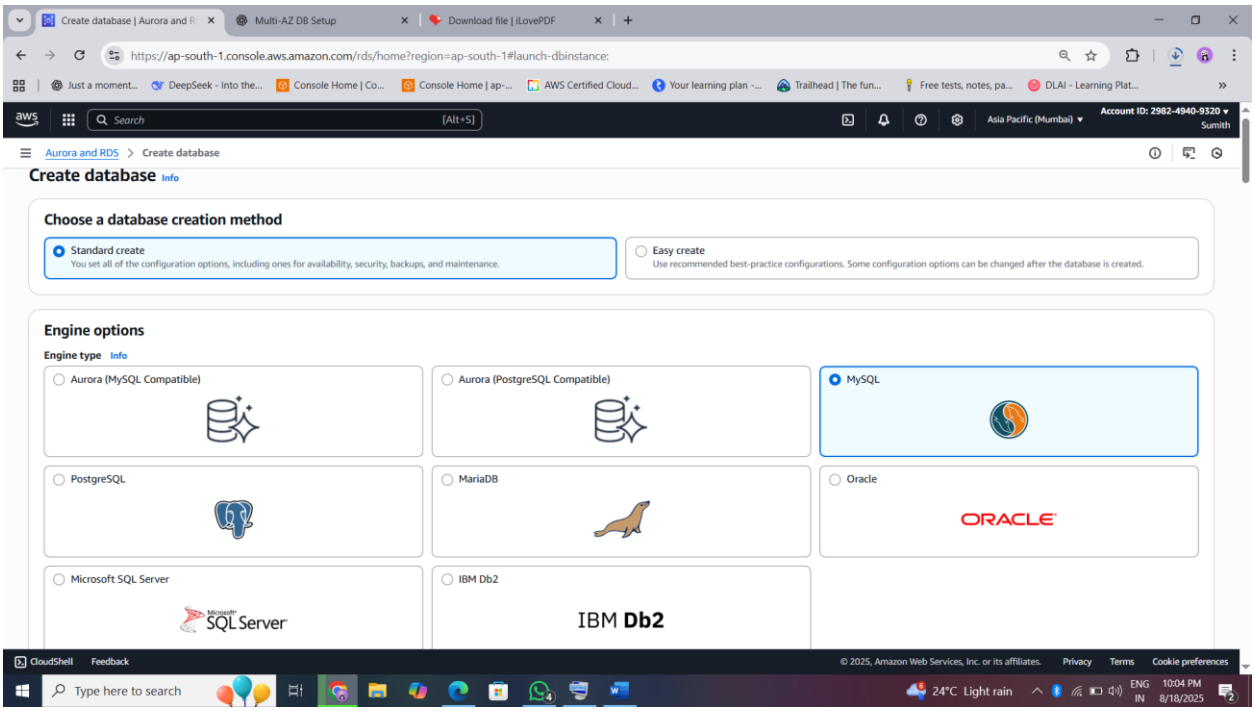
Configure **RDS automated backups** with cross-zone replication, retention policy, and verify restoration.

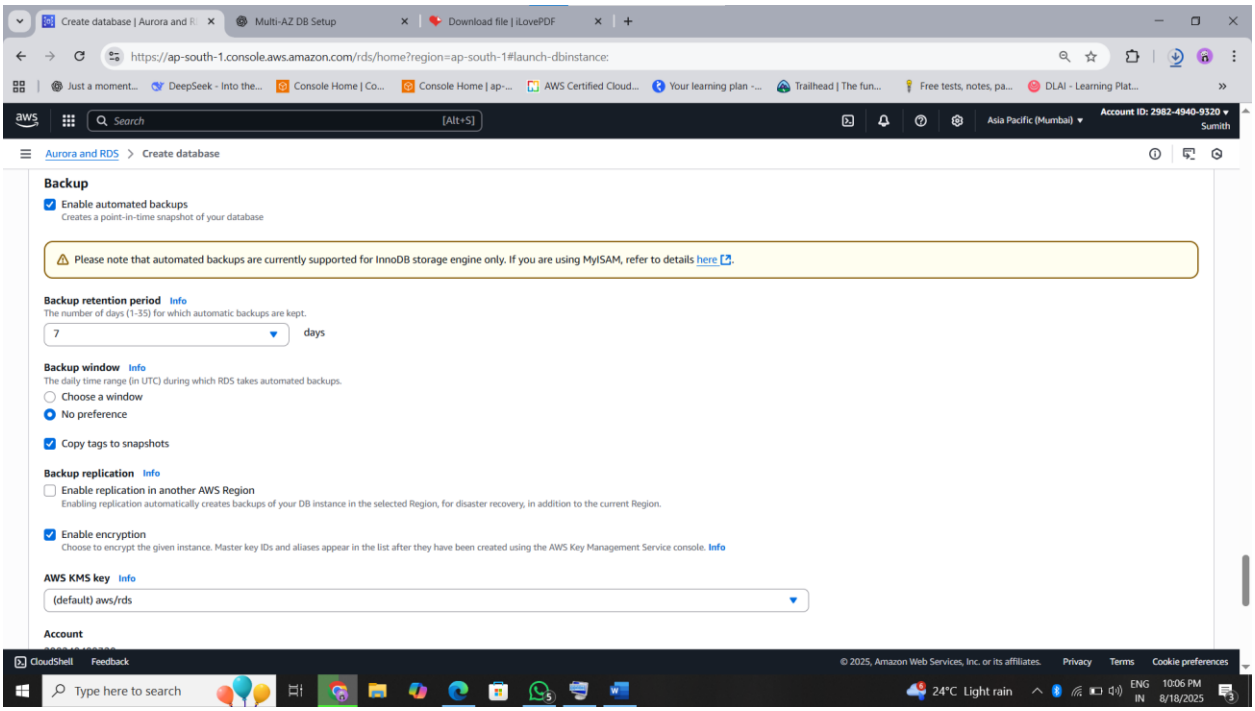
Steps

1. Enable Automated Backups (7-Day Retention)

- Go to **AWS Console → RDS → Databases**.
- Select your DB → **Modify**.
- Under **Backup**:
  - Set **Automated backups = Enabled**.
  - Retention period = **7 days** (can be 1–35).
  - Enable **Copy tags to snapshots** (optional for cost tracking).
- Save & **Apply immediately** (or during next maintenance window).

Automated backups will now create **daily snapshots + transaction logs**.

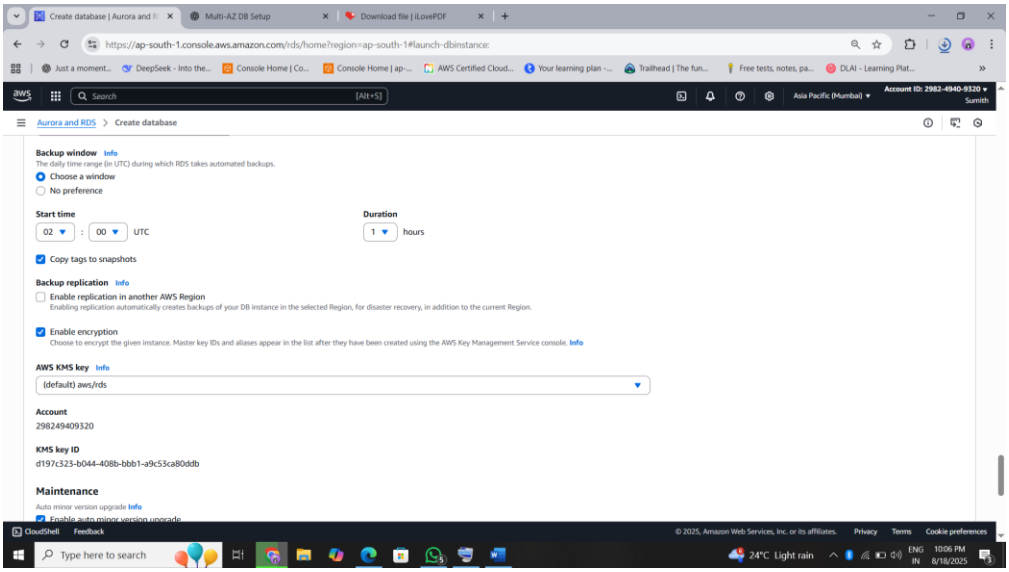




---

2. Configure Backup Window

- In the **Modify DB Instance** page:
  - Find **Preferred backup window**.
  - Example: **02:00–03:00 UTC** → backups run daily at 2 AM.
- Choose a **low-traffic time** to reduce performance impact.



---

3. Cross-Zone / Cross-Region Replication

- Automated backups are already stored in **S3 across multiple AZs** (built-in durability).
- For **cross-region copy**:
  - Go to **RDS → Automated backups → Manage cross-Region automated backups**.
  - Enable replication to another region (extra cost).
- This ensures DR (Disaster Recovery) if a region fails.

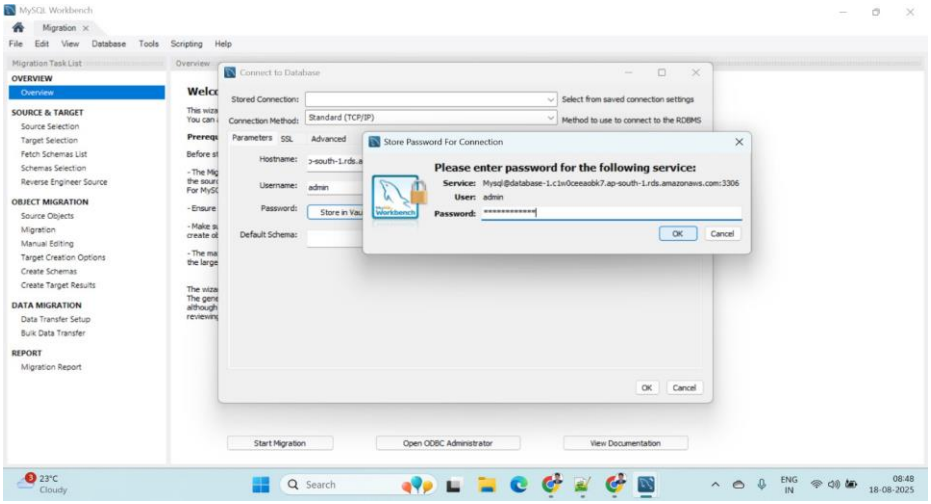
---

4. Test Restoration

- Go to **RDS → Automated backups**.
- Pick a restore point from the last 7 days.
- Click **Restore to point in time** → choose **latest restorable time**.
- Launch as a **new DB instance** (restore-test-db).
- Connect to it → verify that data is intact.

Quick Demo SQL Test (Optional in Lab)

1. Insert dummy row in main DB.
2. INSERT INTO orders VALUES (101, 'Laptop', 50000);
3. Wait for backup window to complete.
4. Delete row.
5. DELETE FROM orders WHERE id = 101;
6. Restore from automated backup.
7. Verify that row 101 is present again.


MySQL Workbench
Migration
File Edit View Database Tools Scripting Help
Migration Task List
OVERVIEW
Overview
SOURCE & TARGET
Source Selection
Target Selection
Fetch Schemas List
Schemas Selection
Reverse Engineer Source
OBJECT MIGRATION
Source Objects
Migration
Manual Editing
Target Creation Options
Create Schemas
Create Target Results
DATA MIGRATION
Data Transfer Setup
Bulk Data Transfer
REPORT
Migration Report
Connect to Database
Stored Connection:
Select from saved connection settings
Connection Method:
Standard (TCP/IP)
Method to use to connect to the RDBMS
Parameters
SSL
Advanced
Hostname:
Username:
admin
Password:
Store in Vau
Default Schema:
Store Password For Connection
Please enter password for the following service:
Service: Mysql@database-1.c1w0ceeaobk7.ap-south-1.rds.amazonaws.com:3306
User: admin
Password:
OK
Cancel
Start Migration
Open ODBC Administrator
View Documentation
23°C
Cloudy
Search
ENG
IN
08:48
18-08-2025

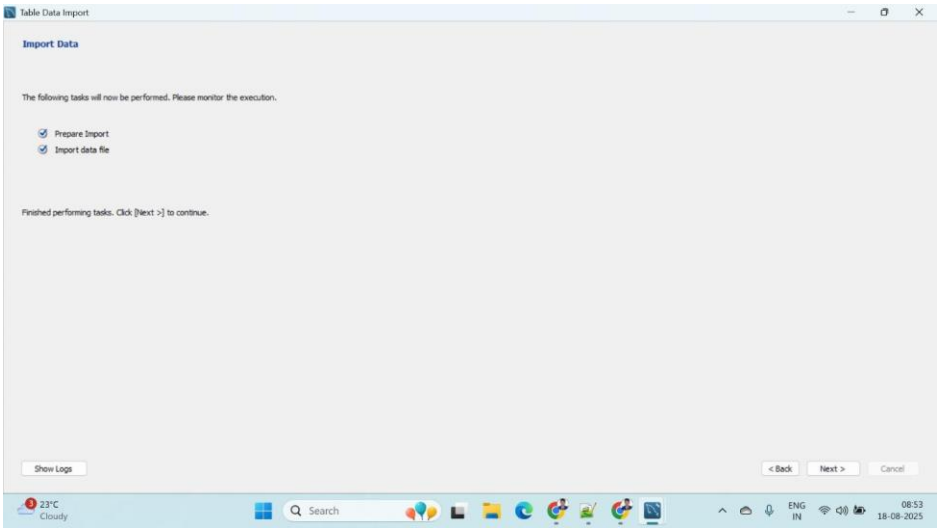


Table Data Import
Import Data
The following tasks will now be performed. Please monitor the execution.
Prepare Import
Import data file
Finished performing tasks. Click [Next >] to continue.
Show Logs
< Back
Next >
Cancel
23°C
Cloudy
Search
ENG
IN
08:53
18-08-2025

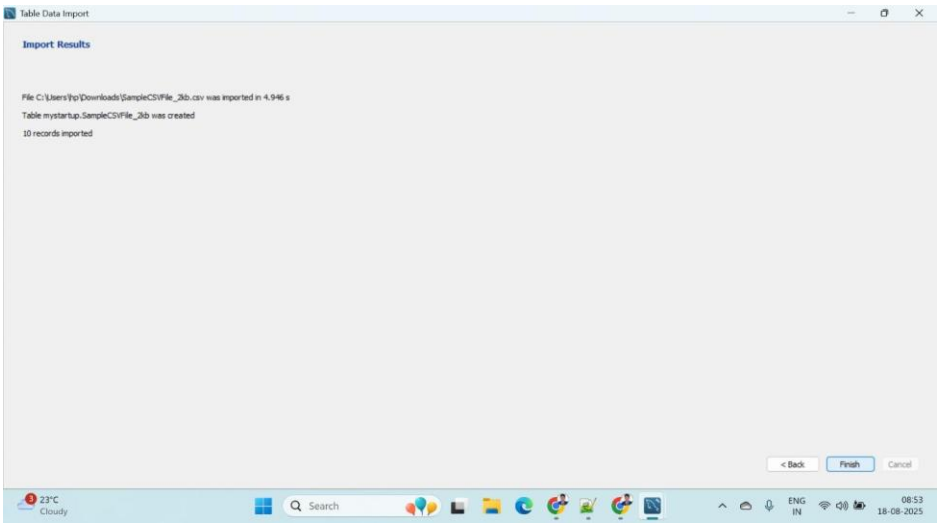
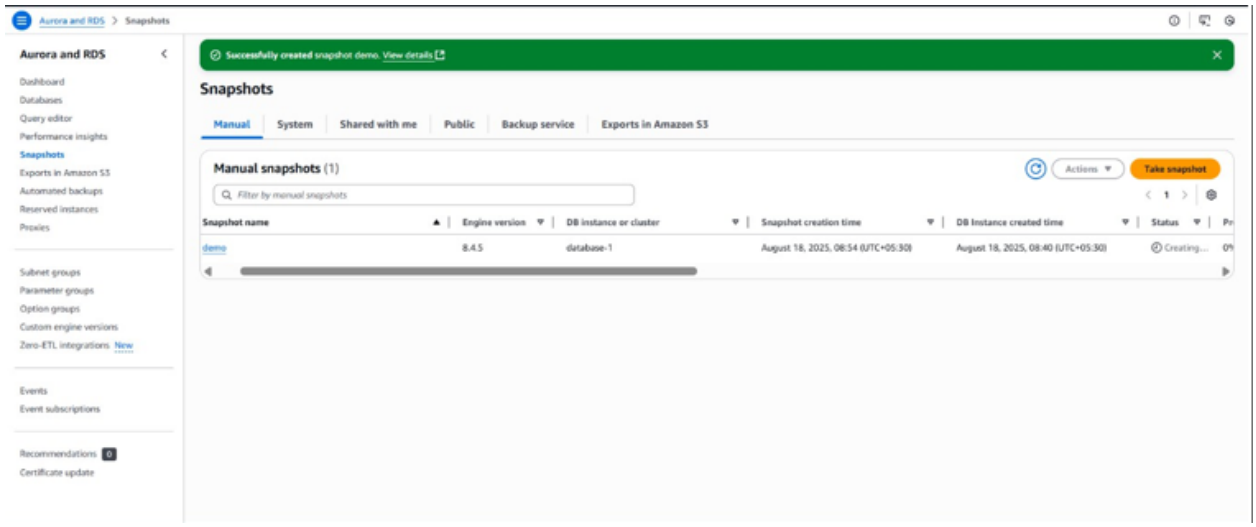


Table Data Import
Import Results
File C:\Users\hp\Downloads\SampleCSV\File_2kb.csv was imported in 4.946 s
Table mystartup.SampleCSV\File_2kb was created
10 records imported
< Back
Finish
Cancel
23°C
Cloudy
Search
ENG
IN
08:53
18-08-2025



Difference Recap

- **Automated backup** = Daily + logs (can do PITR).
- **Snapshot** = On-demand, manual, static.
- **Cross-region backup** = Extra replication for DR.

(note: full screen shots were not took because of internet issues with aws console)

Task 4: Multi-AZ Deployment Testing

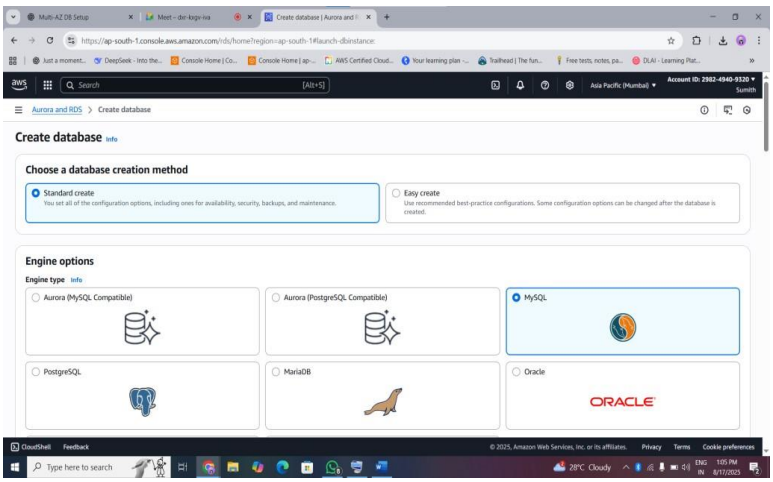
Objective

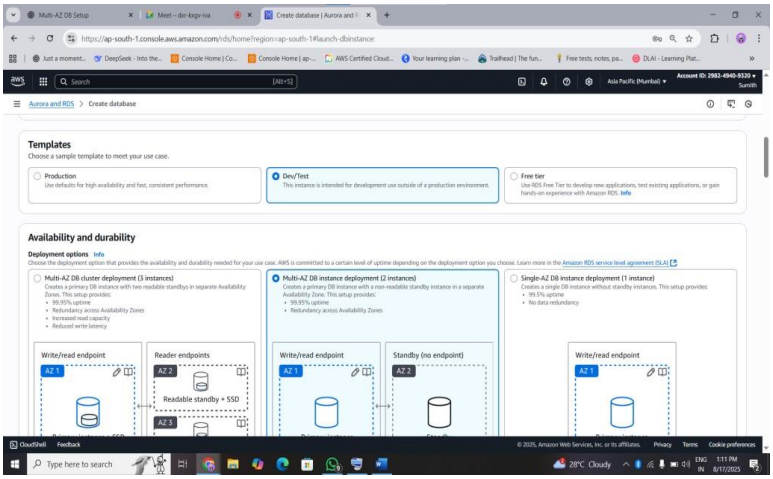
Validate that **Multi-AZ RDS** provides automatic failover in case of a primary instance failure.

Steps

1. Launch Multi-AZ RDS Instance

- Go to **AWS Console → RDS → Create database**.
- Engine: e.g., **MySQL 8.0** (or Postgres).
- Deployment type: **Multi-AZ DB Instance** (not Multi-AZ cluster).
- Instance class: db.t3.micro (for free/low-cost testing).
- Storage: GP3 or GP2.
- Ensure **Multi-AZ** is checked → AWS will create **1 primary + 1 standby replica** in a different AZ.
- Connect via **writer endpoint** (not tied to a single AZ).

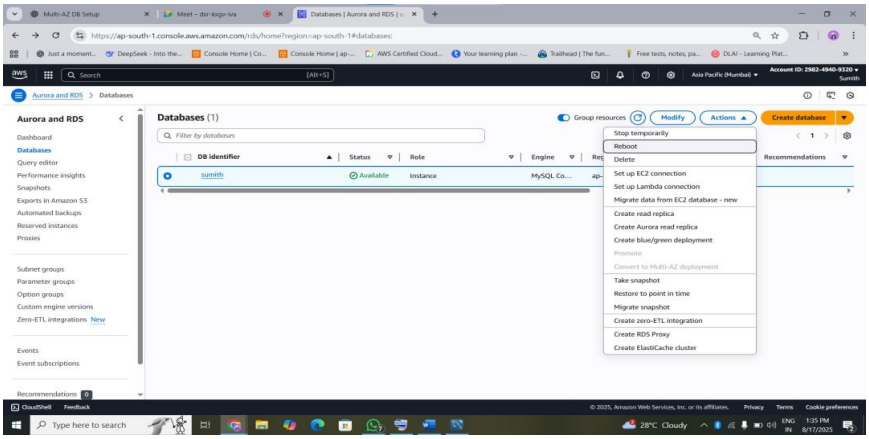


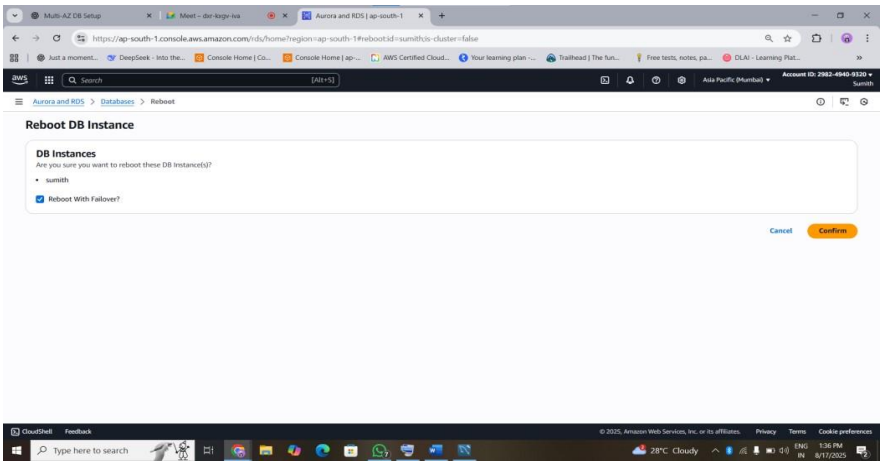


---

2. Simulate Failure

- In **RDS → Databases → your instance → Actions**:
  - Select **Reboot with failover**.
  - AWS will force a **primary → standby** switch.
- Alternatively, you can:
  - Stop the AZ where the primary lives (simulated failure).
  - Or modify instance class to trigger maintenance failover.





3. Observe Automatic Failover

- During failover (~30–60 seconds downtime):
  - Application connections may briefly fail.

    - RDS writer endpoint remains the **same DNS name**, but points to the **new primary**.
- Check in logs:
    - SHOW VARIABLES LIKE 'hostname'; before and after failover → hostname changes.
    - Or run:
    - SELECT @@hostname;

→ It should switch to the standby instance.

---

4. Validate Recovery

- Reconnect to DB via **writer endpoint**.
- Verify that:
    - Database is available.
    - Data consistency is preserved.
- Optionally, monitor **CloudWatch → RDS metrics**:
    - Failover events
    - DatabaseConnections
    - ReplicaLag

Expected Outcome

- Failover time: **30–60 seconds** (can be slightly longer depending on workload).
- No manual changes required in the app connection string.
- Zero data loss (since standby is synchronous).

Note: Multi-AZ ≠ Read Scaling (that's where **Read Replicas** help). Multi-AZ is purely for **HA (High Availability) & DR**.

(note: full screen shots were not took because of internet issues with aws console)

Task 5: Compare Blue-Green vs. Rolling Deployments

Blue-Green Deployment

How it works:

Maintain two identical environments — Blue (current) and Green (new). Switch traffic between them.

Pros:

- Zero downtime (instant traffic switch).
- Easy rollback (switch back to Blue).
- Safer schema changes (you test Green first).
- Works well with **RDS Blue/Green**, **Elastic Beanstalk**, **ECS**, **CodeDeploy**.

Cons:

- Double infrastructure cost (running 2 environments).
- DNS/traffic switch may briefly cause cache inconsistencies.
- Not ideal for very large stateful workloads (data drift risk).

Rolling Deployment

How it works:

Gradually update servers/pods one at a time until all run the new version.

Pros:

- Lower cost (only one environment).
- Smooth transition (users see updates progressively).
- Supported by **ECS, EKS, CodeDeploy, Auto Scaling groups**.

Cons:

- Longer rollout time.
- Harder rollback (need to redeploy old version to all).
- Mixed versions run temporarily (compatibility risks).

AWS Services That Support Blue-Green

- **RDS Blue/Green Deployments** (DB upgrades, schema changes).
- **Elastic Beanstalk** (environment swap).
- **ECS / EKS** (blue-green via CodeDeploy/Service Mesh).

- **AWS Lambda** (via versions & aliases with traffic shifting).
- **CloudFormation Change Sets** (staged stack updates).

Task 6: Encryption & Compliance

Scenario: Client-Side Encryption for RDS

Your company requires **encryption before data leaves the client** (not just at-rest in RDS).

---

1. Implement Encryption using AWS KMS

- **Option A: RDS Native Encryption (PaaS)**
  - Enable **RDS encryption at creation** (uses AWS KMS).
  - Data at rest: Encrypted in storage, logs, snapshots, backups.
  - In-transit: Use **TLS/SSL** between client ↔ DB.
- **Option B: Client-Side Encryption (App Level)**
  - Use AWS KMS SDK or AWS Encryption SDK.
  - Application encrypts data before sending to RDS.
  - Example:
  - from aws_encryption_sdk import encrypt, decrypt
  -
  - ciphertext, header = encrypt(
  - source=b"SensitiveData",
  - key_arn="arn:aws:kms:region:acct-id:key/key-id"
  -)
  - Only ciphertext is stored in RDS; decryption happens client-side.

---

2. Compare RDS Encryption vs. EC2 Self-Managed

Feature	RDS Encryption (KMS, PaaS)	EC2 Self-Managed DB (Custom)
Ease	One click at DB creation	You manage OS, DB, disk encryptio
At-Rest Encryption	Automatic (storage, backups, replica	Use EBS encryption / custom KMS
In-Transit Encryption	SSL/TLS supported	You configure SSL certs manually
Client-Side Encryption	Optional (app-level KMS/SDK)	Optional, more flexibility
Compliance (HIPAA, PCI, GDPR)	Built-in, AWS-managed keys	More audit/control but more effor
	cluded with RDS pricing + KMS reque	Higher (infra + ops overhead)
ity	d to AWS RDS features	ntrol over DB engine, patches

✅ **Conclusion**

- If compliance requires **full client-side control**, use **KMS SDK + client-side encryption**.
- For most enterprises, **RDS KMS encryption** + **SSL in-transit** meets compliance (PCI, HIPAA).
- EC2 self-managed DB is only needed if you require **non-standard encryption libraries** or on-prem/AWS hybrid setups.

Task 7: Cost Analysis for Backups

What matters for price

- **Automated backups & manual snapshots** are billed as **backup storage** (GiB-month).
- You get **free backup storage up to the size of your provisioned DB storage** (per Region, per account). Anything above that is billed (automated + manual snapshots both count toward usage). Amazon Web Services, Inc.+1Amazon Web Services, Inc.
- Public list price for additional RDS backup storage is **about $0.095 per GiB-month** (varies slightly by Region; Calculator will show your Region's rate). Amazon Web Services, Inc.+1

How to model it in the AWS Pricing Calculator

1. Open the **AWS Pricing Calculator** → **Create estimate** → **Amazon RDS**. calculator.aws
2. Pick your Region (e.g., **Asia Pacific (Mumbai)**).
3. Under **Backup storage**, enter your expected **GB of backup data stored** and **how long** it's retained.
4. **Important**: If you want a **conservative (upper-bound)** estimate, don't subtract the free allocation. For a **realistic** estimate, subtract your **free allocation = provisioned DB storage (GiB-month)** from the total backup storage you model.

Quick calculations (so you have numbers right now)

Using $0.095 per GiB-month list price. Adjust slightly per your Region in the Calculator. Amazon Web Services, Inc.

A) Automated backups: 1 TB for 35 days

- 1 TB = **1024 GiB**
- Prorated GiB-months: **1024 × (35/30) = 1,194.67 GiB-months**
- **Cost (gross) ≈ 1,194.67 × $0.095 = $113.49**

If you want to account for **free allocation**, subtract your DB's provisioned size (GiB-months) from 1,194.67:

- If DB is **100 GiB** → billable ≈ 1,094.67 GiB-months → **$103.99**
- If DB is **500 GiB** → billable ≈ 694.67 GiB-months → **$65.99**
- If DB is **1,024 GiB** → billable ≈ 170.67 GiB-months → **$16.21**

Why can there still be a charge even when DB=1 TB? Because we modeled **35 days** of backup storage; total retained backup volume can exceed the free pool. Actual automated backups are **incremental**, so your real usage can be **lower** if daily change rate is small. AWS Documentation

B) Manual snapshot: 1 TB retained for 6 months

- GiB-months: **1024 × 6 = 6,144 GiB-months**
- **Cost (gross) ≈ 6,144 × $0.095 = $583.68**
- Remember: manual snapshots **persist until deleted** (no auto-purge). N2W Software

---

Tips & gotchas to include in your estimate

- **Region**: rates vary slightly—always set the Calculator Region to match your RDS Region. Amazon Web Services, Inc.
- **Free allocation pooling**: Free backup storage equals the **sum of provisioned storage of all RDS DBs in the Region**; automated backups **and** manual snapshots both consume that pool. If your pool is already used by other DBs/snapshots, the billable amount goes up. Amazon Web Services, Inc.
- **After instance deletion**: you're still charged for **snapshots** you keep. Amazon Web Services, Inc.
- **Cross-Region copies/automated cross-Region backups** add **data transfer + destination storage** charges; model them separately if you use DR. Amazon Web Services, Inc.

Deliverable template (what to put into the Calculator share)

- **Workload 1:** "RDS backup storage — automated backups"
  - Quantity: **1,024 GiB**
  - Retention: **35 days**
  - **Note**: Adjust down by free allocation in comments if desired.
- **Workload 2:** "RDS backup storage — manual snapshot"
  - Quantity: **1,024 GiB**
  - Retention: **6 months**

Task 8: Disaster Recovery Drill

🎯 **Objective**

Validate that you can recover an RDS database in another Region in case of an AZ or Regional outage.

🔷 **Steps**

1. Create a snapshot in Region A

- Go to **RDS Console → Databases → Select your DB instance**.
- Click **Actions → Take snapshot**.
- Name it clearly (e.g., drill-snapshot-aug18).
- Wait until status = available.

---

2. Replicate snapshot to Region B

- From the **Snapshots** page, select the snapshot.
- Click **Actions → Copy snapshot**.
- Choose **Destination Region (Region B)**.
- (Optional) Enable **KMS encryption** in destination if required for compliance.
- Monitor until the snapshot in Region B shows status available.

💡 **Notes**

- Cross-Region copy incurs **data transfer + storage costs**.
- You can automate this using **AWS Backup** or **RDS cross-Region automated backups** if frequent replication is required.

---

3. Restore database in Region B

- In Region B, go to **Snapshots → Select copied snapshot → Actions → Restore snapshot**.
- Choose new DB instance settings:
  - Engine version (same as snapshot).
  - Instance class (e.g., db.t3.medium).

    - VPC, Subnet group, Security groups.
- Launch the instance.

✅ At this point, you've recovered your DB in Region B.

🔷 **Validation**

- Connect to the new DB endpoint in Region B (via MySQL Workbench, psql, or CLI).
- Run queries to confirm your data exists.
- Optionally, run an **application failover test** by pointing your app to the Region B DB endpoint.

🔷 **Optional Enhancements**

- **Automated DR**: Use **RDS cross-Region automated backups** (Aurora has Global Database for near real-time).
- **Testing frequency**: Run this drill quarterly.
- **Failback plan**: After Region A recovers, copy snapshot back or re-enable replication.